Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА №1

Изучение методов защиты программного обеспечения

по дисциплине
«Методы защиты программного обеспечения»

Выполнил
студент гр. 5151001/40202 Волошкевич М.А.

Руководитель
ассистент Ерастов В.О.

Санкт-Петербург – 2026

1 ЦЕЛЬ РАБОТЫ

Изучение методов защиты программного обеспечения, способов анализа защищённого исполняемого файла и преодоления реализованных механизмов защиты.

2 ЗАДАЧИ

В рамках лабораторной работы необходимо реализовать программу типа crackme, добавить в неё методы защиты, исследовать бинарный файл, выполнить обход отдельных проверок патчингом и изучить упаковку исполняемого файла. В отчёте рассматриваются исходная и защищённая версии программы, назначение внедрённых механизмов и результаты анализа дизассемблированного кода.

3 ХОД РАБОТЫ

3.1 Описание исходной программы

Исходная программа написана на языке C. Она читает пароль из файла `password.txt` и сравнивает первые девять символов с заданной строкой `123456789`. При совпадении создаётся серийный номер формата `KEY$xxxxxxxxxx$`, содержащий десять случайных буквенно-цифровых символов, и записывается в `serial.txt`. После успешной проверки программа также принимает целые числа из стандартного ввода и вычисляет элементы последовательности, формируемой умножением предыдущих значений на 3, 5 и 7.

При неверном пароле программа выводит `Wrong pass` и завершает работу без создания серийного номера и выполнения основного цикла вычислений. Файлы `password.txt` и `serial.txt` должны находиться в рабочем каталоге программы.

В исходном варианте проверка пароля непосредственно выражена вызовом `strncmp(pass, "123456789", 9)`. Генерация серийного номера и полезная вычислительная функция `give_n_sequence` не защищены дополнительными проверками целостности или антиотладочными механизмами.

3.2 Основные изменения в защищённой версии

В защищённой версии проверка пароля отделена от места её вызова. Используются функции `check_impl`, `real_check` и `real_check_trampoline`, а функция `patch_check` изменяет машинный код в памяти процесса: в точку `patch_point_impl` записывается относительный переход на trampoline, который передаёт управление `real_check`.

Строковые константы, включая имена файлов, сообщения и части серийного номера, хранятся в XOR-зашифрованном виде и восстанавливаются функцией `xor_str`. В программе присутствуют проверки целостности участка исполняемого кода через CRC32, вызов `ptrace`, проверка временных ограничений и проверка

состояния ASLR. Проверки интегрированы в ход выполнения программы, поэтому изменение отдельных переходов может повлиять на дальнейшую работу.

3.3 Анализ проверки пароля и бинарный патчинг

В предоставленном дизассемблерном фрагменте вызов `strncmp` расположен по адресу `0x555555555B3D`, а после него выполняется `test eax, eax`. Следующий условный переход в оригинальном потоке управления определяет, равны ли сравниваемые строки. В исследуемом варианте переход заменён безусловным `jmp` на адрес `0x555555555B50`.

Такое изменение заставляет выполнение продолжаться по ветви, предназначенной для успешного результата проверки, независимо от значения, возвращённого `strncmp`.



Рисунок 1 – Проверка пароля в дизассемблере до и после патчинга

3.4 Анализ проверок целостности и среды выполнения

В листинге `main` вычисляется CRC32 для диапазона адресов между метками `loc_555555555DBA` и `loc_555555556765`. Полученное значение сравнивается с

константой `0xC8ADB734`. В исходном фрагменте условный переход после `cmp` заменён на безусловный `jmp`, благодаря чему программа переходит к продолжению выполнения, минуя ветвь вывода предупреждения `[ALERT]`.

Далее вызывается `ptrace`, после чего возвращаемое значение сравнивается с `-1`. В предоставленном листинге переход после сравнения заменён на безусловный переход к дальнейшему коду. Аналогично показаны изменения ветвлений для проверки временного ограничения и проверки флага ASLR. Эти фрагменты демонстрируют, что проверка защиты может быть найдена по вызову системной функции, сравнению результата и условному переходу, ведущему к диагностическому сообщению или завершению программы.

В другом участке кода проверка `check_impl` выполняется перед созданием и записью серийного номера, а также в цикле программы. Это позволяет проверять условие не только в единственной начальной точке, но и повторно во время работы.



Рисунок 2 – Фрагменты дизассемблированного кода: CRC32, ptrace и проверки среды

3.5 Обфускация строк и изменение потока управления

XOR-шифрование строк скрывает в секции данных исходные текстовые значения. Перед использованием соответствующая строка восстанавливается функцией `xor_str`; поэтому поиск обычных строковых констант в бинарном файле может не дать ожидаемого результата. При анализе необходимо проследить аргументы вызова функции расшифрования и момент, когда восстановленная строка передаётся библиотечной функции.

В защищённой версии присутствуют несколько точек вызова `check_impl`, а также переходы, изменённые патчингом. Такой подход усложняет анализ по одному участку кода, поскольку необходимо учитывать все места проверки и связи между ними. Самомодификация реализована через изменение прав доступа страницы памяти с `mprotect` и запись машинных байтов в `patch_point_impl`; затем права страницы возвращаются к режиму чтения и исполнения.

Требование методички о трёх методах противодействия дизассемблированию, четырёх методах обнаружения отладчика и трёх методах обнаружения виртуальной машины следует подтверждать конкретными реализациями в итоговом исходном коде. По предоставленным фрагментам можно непосредственно подтвердить самомодификацию, XOR-обфускацию строк, CRC-проверку, `ptrace`, проверку времени и проверку ASLR; наличие остальных методов нельзя считать доказанным только по приведённым листингам.

3.6 Упаковка исполняемого файла

Для упаковки использовался UPX. В предоставленном выводе каталога присутствуют исходный исполняемый файл `main` размером около 23 KiB и упакованный `main_upx` размером около 12 KiB. По этим округлённым значениям уменьшение размера составляет приблизительно 47,8 %. Это оценка по округлённому выводу `ls -lh`, а не точное измерение размера в байтах.

UPX добавляет загрузочный код, который восстанавливает упакованные данные в памяти и передаёт управление оригинальной точке входа (OEP). Точное значение OEP следует определить при анализе конкретного `main_upx` в дизассемблере или отладчике. По одному имени файла и приведённым фрагментам защищённого бинарника достоверно установить адрес OEP нельзя.

Для сравнения энтропии необходимо измерить оба бинарных файла, а не оценивать значение по размеру или листингу. Энтропия Шеннона для байтов вычисляется по формуле:

$$H(X) = -\sum_{i=0}^{255} p_i \log_2(p_i) \tag{1}$$

где p_i — доля байта со значением i в файле. Максимальное значение составляет 8 бит на байт. Упаковка часто повышает энтропию упакованной области, однако фактические значения для `main` и `main_upx` нужно получить измерением на исходных файлах.

Таблица 1 – Сравнение характеристик исходного и упакованного файлов

Параметр	**main**	**main_upx**
Размер по `ls -lh`	около 23 KiB	около 12 KiB
Размер в байтах	измерить	измерить
Энтропия, бит/байт	измерить	измерить
Точка входа	из `readelf`	исследовать OEP

4 ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ

4.1 Постановка задачи

В участке самомодифицирующегося кода необходимо реализовать сравнение серийного номера, полученного на основе пароля, с эталонным значением, после чего обойти эту проверку известными методами реверс-инжиниринга и отладки.

4.2 Реализация проверки серийного номера

Серийный номер вычисляется из пароля функцией `serial_from_pass`: к паролю (без завершающих символов перевода строки) применяется CRC32, и результат форматируется в виде `KEY$XXXXXXXX$`. Для верного пароля «123456789» значение CRC32 равно `0xCBF43926`, поэтому эталонный серийный номер — `KEY$CBF43926$`.

Эталонное значение не хранится в открытом виде: строка `str_reference_serial` записана в секции данных в XOR-закодированном виде и расшифровывается в момент проверки тем же ключом, что и остальные строки программы.

Сравнение выполняется в функции `real_check`, то есть именно в участке самомодифицирующегося кода: управление в неё передаётся только после того, как `patch_check` перезапишет пятибайтовую заглушку `mov eax, 0` в `patch_point_impl` инструкцией `jmp` на trampoline и далее на `real_check`. До самомодификации сравнение серийного номера недостижимо, а его код невозможно корректно увидеть ни в статическом дизассемблере до запуска, ни в памяти процесса после запуска без учёта патча.

Листинг 1 – Функции вычисления и проверки серийного номера

```
void serial_from_pass(const char* pass, char* out, size_t out_size)
{
    size_t len = strlen(pass);
    while (len > 0 && (pass[len - 1] == '\n' || pass[len - 1] == '\r'))
        len--;
    uint32_t hash = calculate_crc32((const uint8_t*)pass, len);
    snprintf(out, out_size, "KEY$%08X$", hash);
}

int real_check(const char* a, const char* b)
{
    __asm__ __volatile__("cmpl $0, %[g]\n\t"
                         "jz   1f\n\t"
                         ".byte 0xE8, 0x11, 0x22, 0x33, 0x44\n\t"
                         "1:\n\t"
                         :
                         : [g] "m"(g_opaque)
                         : "memory");

    char derived[16];
    serial_from_pass(a, derived, sizeof(derived));

    char* key = get_key();
    char* reference = xor_str(str_reference_serial, key);
    free(key);

    int result = strncmp(derived, reference, 13) == 0;
    free(reference);
    (void)b;
    return result;
}
```

Поскольку изменился код, попадающий в границы блока контроля целостности `block1_start`—`block1_end`, эталонное значение CRC32 пересчитано: `0xc8adb734` → `0x1c9266bd`. Новое значение получено запуском неподменённой версии программы, которая при срабатывании проверки выводит фактическую контрольную сумму.

Результаты работы проверки приведены в таблице Таблица 2.

Таблица 2 – Результаты проверки серийного номера

Пароль	**Результат**	**serial.txt**
`123456789`	`succsess`	создаётся
`abcdefghi`	`Wrong pass`	не создаётся

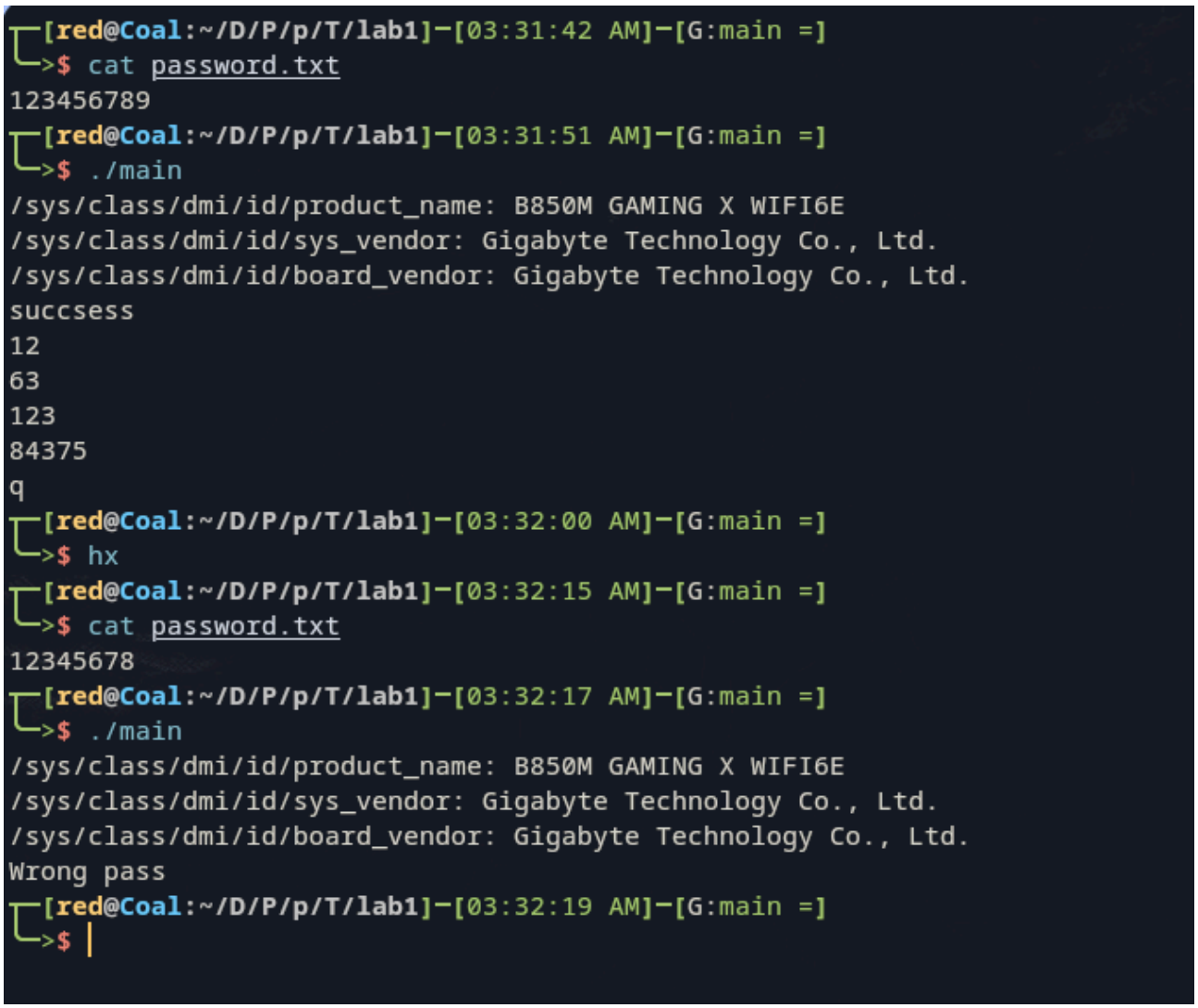


Рисунок 3 – Работа проверки серийного номера при верном и неверном пароле

4.3 Обход проверки в IDA

4.3.1 Статический анализ проверки

Анализ выполняется в IDA Pro на актуальном бинарнике (база данных создаётся заново после пересборки). Символы в файле не вырезаны, поэтому в окне Functions видны все пользовательские функции, а по перекрёстным ссылкам восстанавливается схема защиты:

– `main` вызывает `patch_check`, которая через `mprotect` делает страницу кода записываемой и перезаписывает пятибайтовую заглушку `patch_point_impl` (байты `B8 00 00 00 00`, то есть `mov eax, 0`) инструк-

цией jmp на real_check_trampoline; trampoline восстанавливает %rbp и передаёт управление real_check;

– в real_check (0x1484) вызываются serial_from_pass (0x13E9) и xor_str для расшифровки эталона str_reference_serial (0x5178), затем strncmp (call по адресу 0x150B), а результат оформляется инструкцией sete al по адресу 0x1512.

Само сравнение в статическом листинге напрямую недоступно: до выполнения патча в check_impl видна только заглушка, а явных вызовов real_check в коде нет. Адресат самомодификации определяется анализом ссылок на patch_point_impl и real_check_trampoline.

Эталонное значение восстанавливается без запуска программы: ключ hello, собираемый get_key из трёх глобальных строк, побайтно складывается по модулю два с байтами str_reference_serial (\x23\x20\x35\x48\x2C\x2A\x23\x58\x5F\x56\x5A\x53\x48), что даёт KEY$CBF43926$ — серийный номер, соответствующий CRC32 верного пароля.

4.3.2 Бинарный патчинг в IDA

real_check возвращает истину только тогда, когда strncmp вернул ноль. Точечная правка — заменить инструкцию sete al (0F 94 C0) по адресу 0x1512 на mov al, 1 с добивкой nop (B0 01 90): результат сравнения становится константой 1 независимо от содержимого сравниваемых строк. В IDA это выполняется переходом к адресу клавишей G, правкой байт через Edit → Patch program → Change byte… и записью изменений в бинарник командой Edit → Patch program → Apply patches to input file.

Патч не затрагивает блок block1_start—block1_end, охваченный контролем целостности: real_check лежит вне его границ, поэтому CRC32-проверка не срабатывает. Запуск патченного файла с заведомо неверным паролем abcdefghi приводит к выводу succsess и созданию serial.txt.



```
                public real_check
real_check      proc near               ; CODE XREF: real_check_trampoline+1↓j

var_50          = qword ptr -50h
var_48          = qword ptr -48h
var_34          = dword ptr -34h
ptr             = qword ptr -30h
s2              = qword ptr -28h
s1              = byte ptr -20h
var_8           = qword ptr -8

; __unwind {
                push    rbp
                mov     rbp, rsp
                sub     rsp, 50h
                mov     [rbp+var_48], rdi
                mov     [rbp+var_50], rsi
                mov     rax, fs:28h
                mov     [rbp+var_8], rax
                xor     eax, eax
                cmp     cs:g_opaque, 0
                jz      short loc_14B1
                call    near ptr 443336C2h

loc_14B1:                               ; CODE XREF: real_check+26↑j
                lea     rcx, [rbp+s1]
                mov     rax, [rbp+var_48]
                mov     edx, 10h
                mov     rsi, rcx
                mov     rdi, rax
                call    serial_from_pass
                call    get_key
                mov     [rbp+ptr], rax
                mov     rax, cs:str_reference_serial
                mov     rdx, [rbp+ptr]
                mov     rsi, rdx
                mov     rdi, rax
                call    xor_str
                mov     [rbp+s2], rax
                mov     rax, [rbp+ptr]
                mov     rdi, rax        ; ptr
                call    _free
                mov     rcx, [rbp+s2]
                lea     rax, [rbp+s1]
                mov     edx, 0Dh        ; n
                mov     rsi, rcx        ; s2
                mov     rdi, rax        ; s1
                call    _strncmp
                test    eax, eax
                setz    al
                movzx   eax, al
```

Рисунок 4 – Анализ `real_check` в IDA: вызовы `serial_from_pass`, `xor_str` и `strncmp`



Рисунок 5 – Запуск патченного бинарника с неверным паролем

4.4 Выводы по дополнительному заданию

Проверка серийного номера, размещённая в самомодифицирующемся коде, защищает от статического анализа лучше, чем обычное сравнение строк: до выполнения патча ни логика сравнения, ни эталонное значение недоступны аналитику. Тем не менее она обходится средствами IDA: по перекрёстным ссылкам восстанавливается участок самомодификации и место сравнения, эталон расшифровывается, а точечная замена инструкции `sete al` на `mov al, 1` в точке принятия решения делает проверку всегда успешной; аналогично проверка обходится правкой регистров в отладчике IDA. Это подтверждает общий принцип: защита не может опираться на единственный механизм, а решение о допуске всегда принимается в конкретной инструкции, уязвимой к патчингу.

5 ВЫВОДЫ

В ходе работы рассмотрена программа на C, выполняющая проверку пароля, создание серийного номера и вычисление числовой последовательности. Исследованы способы изменения поведения бинарного файла путём замены условных переходов, а также механизмы защиты, присутствующие в защищённой версии: XOR-обфускация строк, CRC32-проверка участка кода, вызов `ptrace`, проверка времени и состояния ASLR, повторные проверки и самомодификация через `mprotect`.

По предоставленным дизассемблерным фрагментам показано, как обнаружить сравнение пароля, найти связанные с проверками ветви отказа и изменить управление выполнением. Также рассмотрена цель упаковки UPX и необходимость сравнивать исходный и упакованный файлы по точным размерам, энтропии и точке входа. Для завершения отчёта требуется добавить результаты измерений и скриншоты, полученные при запуске инструментов на фактических бинарных файлах.

В дополнительном задании проверка серийного номера, вычисляемого из пароля, реализована в участке самомодифицирующегося кода и затем обойдена средствами IDA: статический анализ по перекрёстным ссылкам выявляет участок самомодификации и точку принятия решения, после чего бинарный патч инструкции `sete` делает проверку всегда успешной; альтернативно проверка обходится динамически в отладчике IDA правкой регистров. Результат демонстрирует, что самомодификация кода затрудняет, но не предотвращает анализ.

ПРИЛОЖЕНИЕ 1. ИСХОДНЫЙ КОД ДО ВНЕДРЕНИЯ ЗАЩИТЫ

Файл: main_simple.c.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define ll unsigned long long

void generate_random_string(char* str, int length)
{
  const char charset[] =
    "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
  int charset_size = sizeof(charset) - 1;

  for (int i = 0; i < length; i++)
  {
    int key = rand() % charset_size;
    str[i] = charset[key];
  }
}

char* read_file_to_string(const char* filename)
{
  FILE* file = fopen(filename, "rb");
  if (file == NULL)
  {
    perror("Не удалось открыть файл");
    return NULL;
  }

  fseek(file, 0, SEEK_END);
  long file_size = ftell(file);
```

```
  rewind(file);

  char* buffer = (char*)malloc(file_size + 1);
  if (buffer == NULL)
  {
    perror("Ошибка выделения памяти");
    fclose(file);
    return NULL;
  }

  size_t bytes_read = fread(buffer, 1, file_size, file);
  buffer[bytes_read] = '\0';

  fclose(file);
  return buffer;
}

typedef struct
{
  ll* data;
  int size;
} dynarr;

void init_buffer(dynarr* buf)
{
  buf->size = 1;
  buf->data = malloc(sizeof(ll));
  buf->data[0] = 1;
}

void add_element(dynarr* buf, ll value)
{
  buf->data = realloc(buf->data, (buf->size + 1) * sizeof(ll));
```

```
  buf->data[buf->size++] = value;
}

void compact_buffer(dynarr* buf, int new_start)
{
  int new_size = buf->size - new_start;
  for (int i = 0; i < new_size; i++)
  {
    buf->data[i] = buf->data[new_start + i];
  }

  buf->size = new_size;
  buf->data = realloc(buf->data, buf->size * sizeof(ll));
}

ll give_n_sequence(int n)
{
  dynarr buf;
  init_buffer(&buf);

  int i3 = 0, i5 = 0, i7 = 0;
  int last_compacted = 0;
  int prev_min_used = 0;

  for (int i = 1; i <= n; i++)
  {
    ll next3 = buf.data[i3] * 3;
    ll next5 = buf.data[i5] * 5;
    ll next7 = buf.data[i7] * 7;

    ll next = next3;
    if (next5 < next)
      next = next5;
```

```
    if (next7 < next)
      next = next7;

    add_element(&buf, next);

    if (next == next3)
      i3++;
    if (next == next5)
      i5++;
    if (next == next7)
      i7++;

    int min_unused = (i3 < i5) ? i3 : i5;
    min_unused = (min_unused < i7) ? min_unused : i7;

    if (min_unused != prev_min_used)
    {
      compact_buffer(&buf, min_unused);
      i3 -= min_unused;
      i5 -= min_unused;
      i7 -= min_unused;
      last_compacted += min_unused;
    }

    prev_min_used = min_unused;
  }

  ll result = buf.data[n - last_compacted];
  free(buf.data);
  return result;
}

int main()
```

```
{
  srand(time(NULL));

  char* pass = read_file_to_string("password.txt");

  if (strncmp(pass, "123456789", 9) != 0)
  {
    printf("Wrong pass\n");
    return 0;
  }

  char* serial = (char*)malloc(16);
  serial[0] = 'K';
  serial[1] = 'E';
  serial[2] = 'Y';
  serial[3] = '$';

  generate_random_string(serial + 4, 10);
  serial[14] = '$';
  serial[15] = '\0';

  FILE* f = fopen("serial.txt", "w");
  fputs(serial, f);

  printf("succsess\n");
  fclose(f);
  free(serial);

  int n;
  while (1)
  {
    if (scanf("%d", &n) != 1)
      break;
```

```
    printf("%llu\n", give_n_sequence(n));
  }

  return 0;
}
```

ПРИЛОЖЕНИЕ 2. ИСХОДНЫЙ КОД ЗАЩИЩЁННОЙ ВЕРСИИ

Файл: `main.c`.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <stdint.h>
#include <sys/ptrace.h>
#include <unistd.h>
#include <sys/personality.h>
#include <cpuid.h>

#define ll unsigned long long

const char* str_open_error =
"\xB8\xF8\xBC\xD9\x4F\xB9\xE6\xBC\xD8\xBF\xD8\xB5\xD7\xBC\xD1\xB9\xE4\xBD"
"\xE0\x4F\xB8\xDB\xBD\xEE\xBF\xD2\xB4\xEC\xBD\xE4\xB9\xE7\xBD\xE0\x4F\xB9"
  "\xE1\xBC\xDC\xBF\xD1\xB5\xD7";
const char* str_memory_error =
"\xB8\xFB\xBD\xE4\xBF\xD0\xB5\xDD\xBC\xD5\xB8\xD5\x4C\xBC\xDD\xB9\xEE\xBC"
"\xD8\xBF\xDD\xB5\xD7\xBC\xDA\xB8\xD8\xBC\xD4\xBE\xE7\x45\xBC\xD3\xBF\xD8"
  "\xB5\xD0\xBD\xE0\xB9\xE7\xBC\xD4";

const char* str_password_file =
  "\x18\x04\x1F\x1F\x18\x07\x17\x08\x42\x1B\x10\x11";
const char* str_serial_file =
"\x1B\x65\x1E\x05\x0E\x04\x4B\x18\x14\x1B";

const char* str_format = "\x4D\x16\x49\x1F\x4A\x1B";
const char* str_print_s = "\x4D\x16";
const char* str_numbers = "\x59\x57\x5F\x58\x5A\x5E\x52\x54\x55";
```

```
const char* str_wrong_pass =
"\x3F\x17\x03\x02\x08\x48\x15\x0D\x1F\x1C\x62";
const char* str_success = "\x1B\x10\x0F\x0F\x1C\x0D\x16\x1F\x66";

const char* str_write = "\x1F";
const char* str_scan_int = "\x4D\x01";
const char* str_print_ll = "\x4D\x09\x6C\x19\x65";

const char* str_serial_prefix = "\x23\x20\x35\x48";
const char* str_serial_suffix = "\x4C";

#include <stdint.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

static volatile int g_opaque = 0;

int real_check(const char* a, const char* b)
{
  __asm__ __volatile__("cmpl $0, %[g]\n\t"
            "jz   1f\n\t"
            ".byte 0xE8, 0x11, 0x22, 0x33, 0x44\n\t"
            "1:\n\t"
            :
            : [g] "m"(g_opaque)
            : "memory");
  return strncmp(a, b, 9) == 0;
}

__asm__(".text\n"
    ".globl real_check_trampoline\n"
    "real_check_trampoline:\n\t"
```

```
    "pop %rbp\n\t"
    "jmp real_check\n");

extern void real_check_trampoline(void);
extern char patch_point_impl;

int check_impl(const char* a, const char* b)
{
  int result;
  __asm__ __volatile__(".globl patch_point_impl\n"
            "patch_point_impl:\n\t"
            ".byte 0xB8, 0x00, 0x00, 0x00, 0x00\n\t"
            : "=a"(result)
            : "D"(a), "S"(b)
            : "memory");
  return result;
}

void patch_check(void)
{
  long pgsz = sysconf(_SC_PAGESIZE);
  uintptr_t addr = (uintptr_t)&patch_point_impl;
  void* page = (void*)(addr & ~((uintptr_t)pgsz - 1));

  if (mprotect(page, (size_t)pgsz, PROT_READ | PROT_WRITE |
PROT_EXEC) != 0)
    return;

  int32_t off = (int32_t)((uintptr_t)&real_check_trampoline - (addr +
5));
  unsigned char patch[5] = {0xE9}; /* E9 = jmp rel32 */
  memcpy(patch + 1, &off, 4);
  memcpy((void*)addr, patch, 5);
```

```
  mprotect(page, (size_t)pgsz, PROT_READ | PROT_EXEC);
}

char* get_key();

char* xor_str(const char* str, const char* key)
{
  int len = strlen(str);
  int key_len = strlen(key);

  char* new = (char*)malloc(len + 1);

  if (new == NULL)
    return NULL;

  for (int i = 0; i < len; i++)
  {
    new[i] = str[i] ^ key[i % key_len];

    if (new[i] == '\0')
      new[i] = str[i];
  }

  new[len] = '\0';

  return new;
}

static long long get_time_ns(void)
{
  struct timespec ts;
```

```
  clock_gettime(CLOCK_MONOTONIC, &ts);

  return (long long)ts.tv_sec * 1000000000LL + ts.tv_nsec;
}

const char* first = "he";

void print_byte_str(const char* str)
{
  int len = strlen(str);
  char* x_str = xor_str(str, "hello");

  for (int i = 0; i < len; i++)
  {
    printf("\\x%02X", (unsigned char)x_str[i]);
  }

  printf("\n");

  free(x_str);
}

uint32_t calculate_crc32(const uint8_t* data, size_t length)
{
  uint32_t crc = 0xFFFFFFFF;
  static uint32_t table[256];
  static int initialized = 0;
  if (!initialized)
  {
    for (uint32_t i = 0; i < 256; i++)
    {
      uint32_t c = i;
      for (int j = 0; j < 8; j++)
```

```
      {
        if (c & 1)
          c = 0xEDB88320 ^ (c >> 1);
        else
          c >>= 1;
      }
      table[i] = c;
    }
    initialized = 1;
  }
  for (size_t i = 0; i < length; i++)
  {
    crc = table[(crc ^ data[i]) & 0xFF] ^ (crc >> 8);
  }
  return crc ^ 0xFFFFFFFF;
}

void generate_random_string(char* str, int length)
{
  char* key = get_key();

  char* charset = xor_str(
"\x09\x07\x0F\x08\x0A\x0E\x02\x04\x05\x05\x03\x09\x01\x02\x6F\x18\x14"
"\x1E\x1F\x1B\x1D\x13\x1B\x14\x16\x12\x24\x2E\x2F\x2B\x2D\x23\x2B\x24"
"\x26\x22\x2E\x20\x21\x21\x27\x35\x3D\x3E\x3C\x3C\x30\x3A\x3B\x37\x31"
    "\x3F\x5C\x5D\x5D\x5B\x51\x59\x5A\x58\x50\x5C",
    key);

  free(key);

  int charset_size = strlen(charset);
```

```
  for (int i = 0; i < length; i++)
  {
    int index = rand() % charset_size;
    str[i] = charset[index];
  }

  free(charset);
}

char* read_file_to_string(const char* filename)
{
  FILE* file = fopen(filename, "rb");

  if (file == NULL)
  {
    char* key = get_key();
    char* message = xor_str(str_open_error, key);
    free(key);

    perror(message);
    free(message);

    return NULL;
  }

  fseek(file, 0, SEEK_END);

  long file_size = ftell(file);

  rewind(file);

  char* buffer = (char*)malloc(file_size + 1);
```

```c
  if (buffer == NULL)
  {
    char* key = get_key();
    char* message = xor_str(str_memory_error, key);

    free(key);
    perror(message);

    free(message);
    fclose(file);

    return NULL;
  }

  size_t bytes_read = fread(buffer, 1, file_size, file);
  buffer[bytes_read] = '\0';
  fclose(file);

  return buffer;
}

const char* second = "ll";

typedef struct
{
  ll* data;
  int size;
} dynarr;

void init_buffer(dynarr* buf)
{
  buf->size = 1;
  buf->data = malloc(sizeof(ll));
```

```
  buf->data[0] = 1;
}

void add_element(dynarr* buf, ll value)
{
  buf->data = realloc(buf->data, (buf->size + 1) * sizeof(ll));
  buf->data[buf->size++] = value;
}

void compact_buffer(dynarr* buf, int new_start)
{
  int new_size = buf->size - new_start;

  for (int i = 0; i < new_size; i++)
  {
    buf->data[i] = buf->data[new_start + i];
  }

  buf->size = new_size;
  buf->data = realloc(buf->data, buf->size * sizeof(ll));
}

ll give_n_sequence(int n, char* pass, char* right_pass)
{
  // Another check in program
  if (strncmp(pass, right_pass, 9) != 0)
  {
    return 0;
  }

  dynarr buf;

  init_buffer(&buf);
```

```
int i3 = 0;
int i5 = 0;
int i7 = 0;

int last_compacted = 0;
int prev_min_used = 0;

for (int i = 1; i <= n; i++)
{
  ll next3 = buf.data[i3] * 3;
  ll next5 = buf.data[i5] * 5;
  ll next7 = buf.data[i7] * 7;

  ll next = next3;

  if (next5 < next)
    next = next5;

  if (next7 < next)
    next = next7;

  add_element(&buf, next);

  if (next == next3)
    i3++;

  if (next == next5)
    i5++;

  if (next == next7)
    i7++;
```

```
    int min_unused = (i3 < i5) ? i3 : i5;
    min_unused = (min_unused < i7) ? min_unused : i7;
    if (min_unused != prev_min_used)
    {
      compact_buffer(&buf, min_unused);

      i3 -= min_unused;
      i5 -= min_unused;
      i7 -= min_unused;

      last_compacted += min_unused;
    }

    prev_min_used = min_unused;
  }

  ll result = buf.data[n - last_compacted];

  free(buf.data);

  return result;
}

const char* third = "o";

int main()
{
  patch_check();
  srand(time(NULL));

  // print_byte_str(str_open_error);
  // print_byte_str(str_memory_error);
  // print_byte_str(str_password_file);
```

```
  // print_byte_str(str_serial_file);
  // print_byte_str(str_format);
  // print_byte_str("123456789");
  // print_byte_str(str_wrong_pass);
  // print_byte_str(str_success);
  // print_byte_str(str_write);
  // print_byte_str(str_scan_int);
  // print_byte_str(str_print_ll);
  // print_byte_str(str_serial_prefix);
  // print_byte_str(str_serial_suffix);

  extern void block1_start();
  extern void block1_end();

  uint8_t* start_ptr = (uint8_t*) && block1_start;
  uint8_t* end_ptr = (uint8_t*) && block1_end;

  size_t block_size = end_ptr - start_ptr;

  uint32_t real_crc = calculate_crc32(start_ptr, block_size);

  uint32_t expected_crc = 0xc8adb734;

  if (real_crc != expected_crc)
  {
    printf("[ALERT] Кусок кода модифицирован %x\n", real_crc);
    return 0;
  }

block1_start:
  __asm__ __volatile__("");

  // Check debugger with ptrace
```

```c
if (ptrace(PTRACE_TRACEME, 0, NULL, NULL) == -1)
{
  printf("Ptrace is not work!\n");
  return 1;
}

// Check by proccess tree
pid_t ppid = getppid();

char path[64];
char cmdline[256];

snprintf(path, sizeof(path), "/proc/%d/cmdline", ppid);

FILE* f_proc = fopen(path, "r");

if (!f_proc)
  return 1;

size_t sz = fread(cmdline, 1, sizeof(cmdline) - 1, f_proc);
fclose(f_proc);

cmdline[sz] = '\0';

if (strstr(cmdline, "gdb") || strstr(cmdline, "lldb") ||
  strstr(cmdline, "strace") || strstr(cmdline, "ltrace") ||
  strstr(cmdline, "rr"))
{
  printf("Debugging tool detected: %s\n", cmdline);
  return 1;
}

// Check by time of execution
```

```
const long long threshold = 100000000;
long long start = get_time_ns();

volatile unsigned long long x = 0;
for (unsigned long long i = 0; i < 1000000; i++)
  x += i;

long long end = get_time_ns();
long long elapsed = end - start;

if (elapsed > threshold)
{
  printf("Time end\n");
  return 1;
}

// Check by ASLR enabled
unsigned long personality_value = personality(0xffffffffUL);

if (personality_value == (unsigned long)-1)
{
  perror("personality");
  return 1;
}

if (personality_value & ADDR_NO_RANDOMIZE)
{
  printf("ASLR disabled\n");
  return 1;
}

// Check CPUID hypervisor present bit
unsigned int eax, ebx, ecx, edx;
```

```
if (!__get_cpuid(1, &eax, &ebx, &ecx, &edx))
  return 1;

if (ecx & (1u << 31))
{
  printf("CPUID\n");
  return 1;
}

// Check BIOS and system info
const char* files[] = {"/sys/class/dmi/id/product_name",
             "/sys/class/dmi/id/sys_vendor",
             "/sys/class/dmi/id/board_vendor"};

const char* vm_names[] = {
  "VMware", "VirtualBox", "KVM", "QEMU", "Microsoft Corporation",
  "innotek"};

char line[256];

for (int i = 0; i < 3; ++i)
{
  FILE* f = fopen(files[i], "r");

  if (!f)
    continue;

  if (fgets(line, sizeof(line), f))
  {
    printf("%s: %s", files[i], line);

    for (int j = 0; j < 6; ++j)
```

```
    {
      if (strstr(line, vm_names[j]))
      {
        printf("Info\n");
        fclose(f);
        return 0;
      }
    }
  }

  fclose(f);
}

FILE* f_virt = fopen("/proc/modules", "r");

if (!f_virt)
  return 1;

char line_virt[512];

const char* modules[] = {"vboxguest", "vmw_balloon",  "vmwgfx",
             "virtio",    "xen_blkfront", "xen_netfront"};

while (fgets(line_virt, sizeof(line_virt), f_virt))
{
  for (int i = 0; i < 6; ++i)
  {
    if (strstr(line_virt, modules[i]))
    {
      printf("Virtualization driver detected: %s", modules[i]);

      fclose(f_virt);
      return 0;
```

```
    }
  }
}

fclose(f_virt);

char* key = get_key();
char* pass_filename = xor_str(str_password_file, key);
char* print_s = xor_str(str_print_s, key);

// Read pass from file
char* pass = read_file_to_string(pass_filename);

free(pass_filename);

if (pass == NULL)
  return 0;

char* right_pass = xor_str(str_numbers, key);

short right = 0;
char* wrong = xor_str(str_wrong_pass, key);
// Fake check
if (check_impl(pass, right_pass))
{
  right = 1;
}

char* serial = (char*)malloc(16);

if (serial == NULL)
  return 0;
```

```
char* prefix = xor_str(str_serial_prefix, key);

serial[0] = prefix[0];
serial[1] = prefix[1];
serial[2] = prefix[2];
serial[3] = prefix[3];

free(prefix);

generate_random_string(serial + 4, 10);

char* suffix = xor_str(str_serial_suffix, key);
serial[14] = suffix[0];
free(suffix);
serial[15] = '\0';

key = get_key();
char* serial_filename = xor_str(str_serial_file, key);

char* write_mode = xor_str(str_write, key);

// Real check
if (!check_impl(pass, right_pass) && (right * (right + 1)) % 2 == 0)
{
  right = 1;
  printf(print_s, wrong);
  return 0;
}

FILE* f = fopen(serial_filename, write_mode);

free(serial_filename);
free(write_mode);
```

```
  if (f == NULL)
  {
    free(serial);
    return 0;
  }

  fputs(serial, f);

  char* success = xor_str(str_success, key);

  printf(print_s, success);
  free(success);

  fclose(f);
  free(serial);

  int n;

block1_end:
  __asm__ __volatile__("");

  while (1)
  {
    // Check in real program
    if (!check_impl(pass, right_pass) && ((7 * right * right) - 1) %
3 != 0)
    {
      return 0;
    }

    // Another fake check
    if (check_impl(pass, right_pass))
```

```
    {
      right = 1;
    }

    char* input_format = xor_str(str_scan_int, key);
    if (scanf(input_format, &n) != 1)
    {
      free(input_format);
      break;
    }

    if (n == 113)
      break;

    free(input_format);

    char* output_format = xor_str(str_print_ll, key);
    printf(output_format, give_n_sequence(n, pass, right_pass));
    free(output_format);
  }

  free(key);
  free(right_pass);
  free(pass);

  return 0;
}

char* get_key()
{
  char* key = (char*)malloc(6);

  if (key == NULL)
```

```
    return NULL;

  sprintf(key, "%s%s%s", first, second, third);

  return key;
}

void marker_end();
```